Chapter 9 ▪ Security

When Java technology first appeared on the scene, the excitement was not about a well-crafted programming language but about the possibility of safely executing applets delivered over the Internet. Obviously, serving executable applets is only practical when the recipients can be sure the code won't wreak havoc on their machines. Security therefore was and is a major concern of both the designers and the users of Java technology. This means that unlike other languages and systems, where security was implemented as an afterthought or as a reaction to break-ins, security mechanisms are an integral part of Java technology.

The security architecture was composed of three parts:

1. Language and virtual machine design features (bounds checking on arrays, no unchecked type conversions, no pointer arithmetic, and so on).
2. A *security manager* that controls what the code can do (file access, network access, and so on).
3. Code signing, whereby code authors can use standard cryptographic algorithms to authenticate Java code. Then, the users of the code can determine exactly who created the code and ensure that the code has not been altered after it was signed.

The first part has been a resounding success. C++ programs are routinely vulnerable to attacks such as buffer overruns, but Java provides strong protection. Unfortunately, the other two parts have not fared as well. The security manager was complex and had a large attack surface. It is being deprecated as of Java 17. The code signing architecture was effectively abandoned with the demise of applets and Java Web Start—two mechanisms for the secure delivery of client-side applications.

In previous editions of this book, the security manager and code signing were the highlights of this chapter, but obviously the need for such coverage has gone away. We will first discuss *class loaders* that check class files for integrity when they are loaded into the virtual machine. I will demonstrate how that mechanism can detect tampering with class files. Then, you'll learn about an authentication framework and cryptographic algorithms which are useful for many security-related operations.

9.1. Class Loaders

A Java compiler converts source instructions into code for the Java virtual machine. The virtual machine code is stored in a class file with a `.class` extension. Each class file contains the definition and implementation code for one class or interface. In the following section, you will see how the virtual machine loads these class files.

9.1.1. The Class-Loading Process

The virtual machine loads only those class files that are needed for the execution of a program. For example, suppose program execution starts with `MyProgram.class`. Here are the steps that the virtual machine carries out:

1. The virtual machine has a mechanism for loading class files—for example, by reading the files from disk or by requesting them from the Web; it uses this mechanism to load the contents of the `MyProgram` class file.
2. If the `MyProgram` class has fields or superclasses of another class type, their class files are loaded as well. (The process of loading all the classes that a given class depends on is called *resolving* the class.)
3. The virtual machine then executes the `main` method in `MyProgram`.
4. If the `main` method or a method that `main` calls requires additional classes, these are loaded next.

The class loading mechanism doesn't just use a single class loader, however. Every Java program has at least three class loaders:

- The bootstrap class loader
- The platform class loader
- The system class loader (sometimes called the application class loader)

The bootstrap class loader loads the platform classes contained in the modules

```
java.base
java.datatransfer
java.desktop
java.instrument
java.logging
java.management
java.management.rmi
java.naming
java.prefs
java.rmi
java.security.sasl
java.xml
```

as well as a number of JDK-internal modules.

There is no `ClassLoader` object corresponding to the bootstrap class loader. For example,

```
StringBuilder.class.getClassLoader()
```

returns `null`.

Prior to Java 9, the Java platform classes were located in a file `rt.jar`. Nowadays, the Java platform is modular, and each platform module is contained in a JMOD file (see **Chapter 9 of Volume I**). The platform class loader loads all classes of the Java platform that are not loaded by the bootstrap class loader.

The system class loader loads application classes from the module path and class path.



Note: Prior to Java 9, an "extension class loader" loaded "standard extensions" from the `jre/lib/ext` directory, and an "endorsed standards override" mechanism provided a way of overriding certain platform classes (including the CORBA and XML implementations) with newer versions. Both of these mechanisms have been removed.

9.1.2. The Class Loader Hierarchy

Class loaders have a *parent/child* relationship. Every class loader except for the bootstrap one has a parent class loader. A class loader will give its parent a chance to load any given class and will only load it if the parent has failed. For example, when the system class loader is asked to load a platform class (say, `java.lang.StringBuilder`), it first asks the platform class loader. That class loader first asks the bootstrap class loader. The bootstrap class loader finds and loads the class, so neither of the other class loaders searches any further.

Some programs have a plugin architecture in which certain parts of the code are packaged as optional plugins. If the plugins are packaged as JAR files, you can simply load the plugin classes with an instance of `URLClassLoader`.

```
var url = new URL("file:///path/to/plugin.jar");
var pluginLoader = new URLClassLoader(new URL[] { url });
Class<?> cl = pluginLoader.loadClass("mypackage.MyClass");
```

Since no parent was specified in the `URLClassLoader` constructor, the parent of the `pluginLoader` is the system class loader. **Figure 9.1** shows the hierarchy.

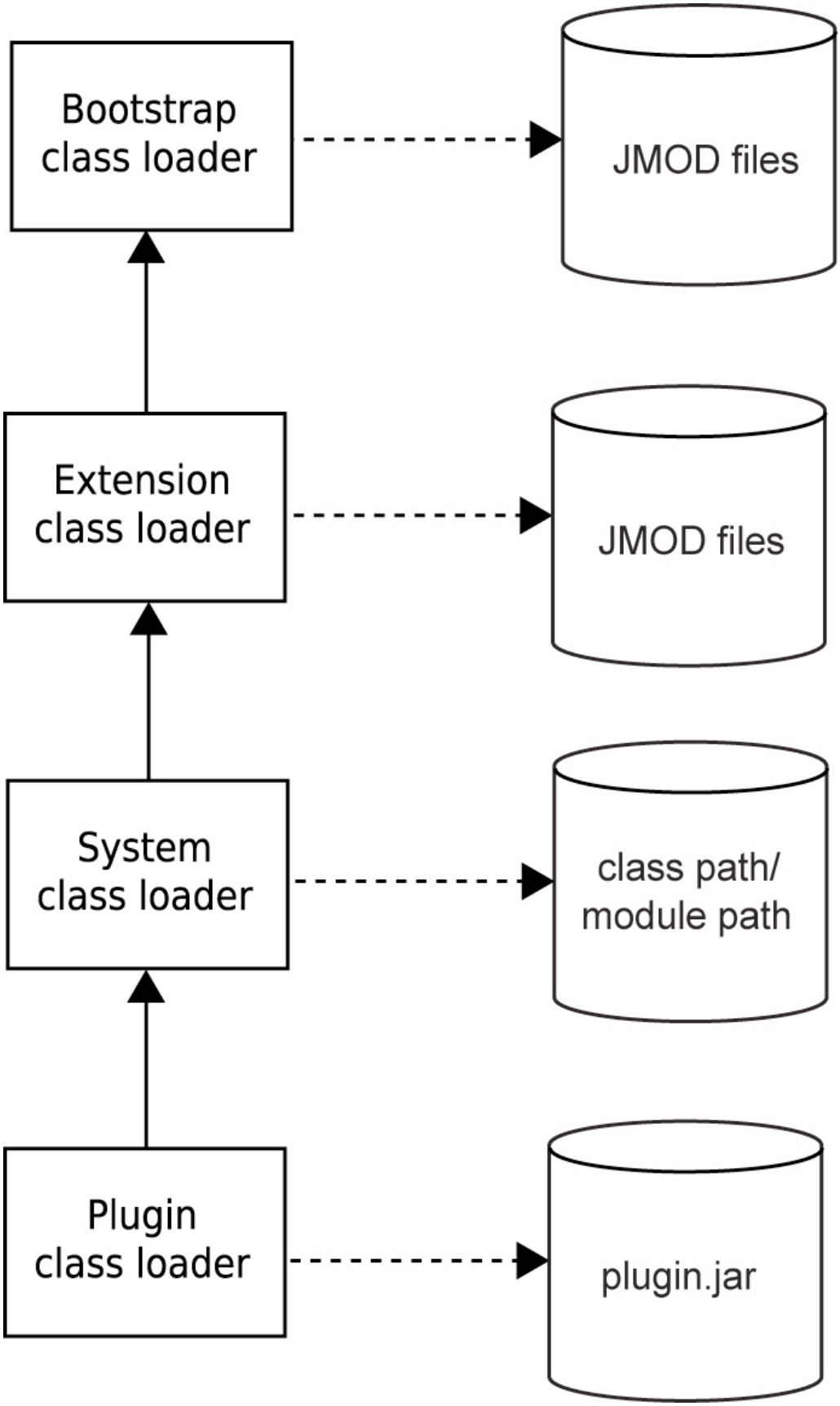


Figure 9.1: The class loader hierarchy



Caution: Prior to Java 9, the system class loader was an instance of `URLClassLoader`. Some programmers used a cast to access the `getURLs` method, or added JAR files to the class path by calling the protected `addURLs` method through reflection. This is no longer possible.

Most of the time, you don't have to worry about the class loader hierarchy. Generally, classes are loaded because they are required by other classes, and that process is transparent to you.

Occasionally, however, you need to intervene and specify a class loader. Consider this example:

- Your application code contains a helper method that calls `Class.forName(classNameString)`.
- That method is called from a plugin class.
- The `classNameString` specifies a class that is contained in the plugin JAR.

The author of the plugin wants the class to be loaded. However, the helper method's class was loaded by the system class loader, and that is the class loader used by `Class.forName`. The classes in the plugin JAR are not visible. This phenomenon is called *classloader inversion*.

To overcome this problem, the helper method needs to use the correct class loader. It can require the class loader as a parameter. Alternatively, it can require that the correct class loader is set as the *context class loader* of the current thread. This strategy is used by many frameworks (such as JAXP and JNDI).

Each thread has a reference to a class loader, called the context class loader. The main thread's context class loader is the system class loader. When a new thread is created, its context class loader is set to the creating thread's context class loader. Thus, if you don't do anything, all threads will have their context class loaders set to the system class loader.

However, you can set any class loader by calling

```
Thread t = Thread.currentThread();
t.setContextClassLoader(loader);
```

The helper method can then retrieve the context class loader:

```
Thread t = Thread.currentThread();
ClassLoader loader = t.getContextClassLoader();
Class<?> cl = loader.loadClass(className);
```



Tip: If you write a method that loads a class by name, it is a good idea to offer the caller the choice between passing an explicit class loader and using the context class loader. Don't simply use the class loader of the method's class.

9.1.3. Using Class Loaders as Namespaces

Every Java programmer knows that package names are used to eliminate name conflicts. There are two classes called `Date` in the standard library, but of course their real names are `java.util.Date` and `java.sql.Date`. The simple name is only a programmer convenience and requires the inclusion of appropriate `import` statements. In a running program, all class names contain their package names.

It might surprise you, however, that you can have two different classes in the same virtual machine that have the same class *and package* name. A class is determined by its full name *and* the class loader. This technique is useful for loading code from multiple sources. For example, an application server uses separate class loaders for each application. This allows the virtual machine to separate classes from different applications, no matter what they are named.

Figure 9.2 shows an example. Suppose an application server loads two different applications, and each has a class called `Util`. Since each class is loaded by a separate class loader, these classes are entirely distinct and do not conflict with each other.

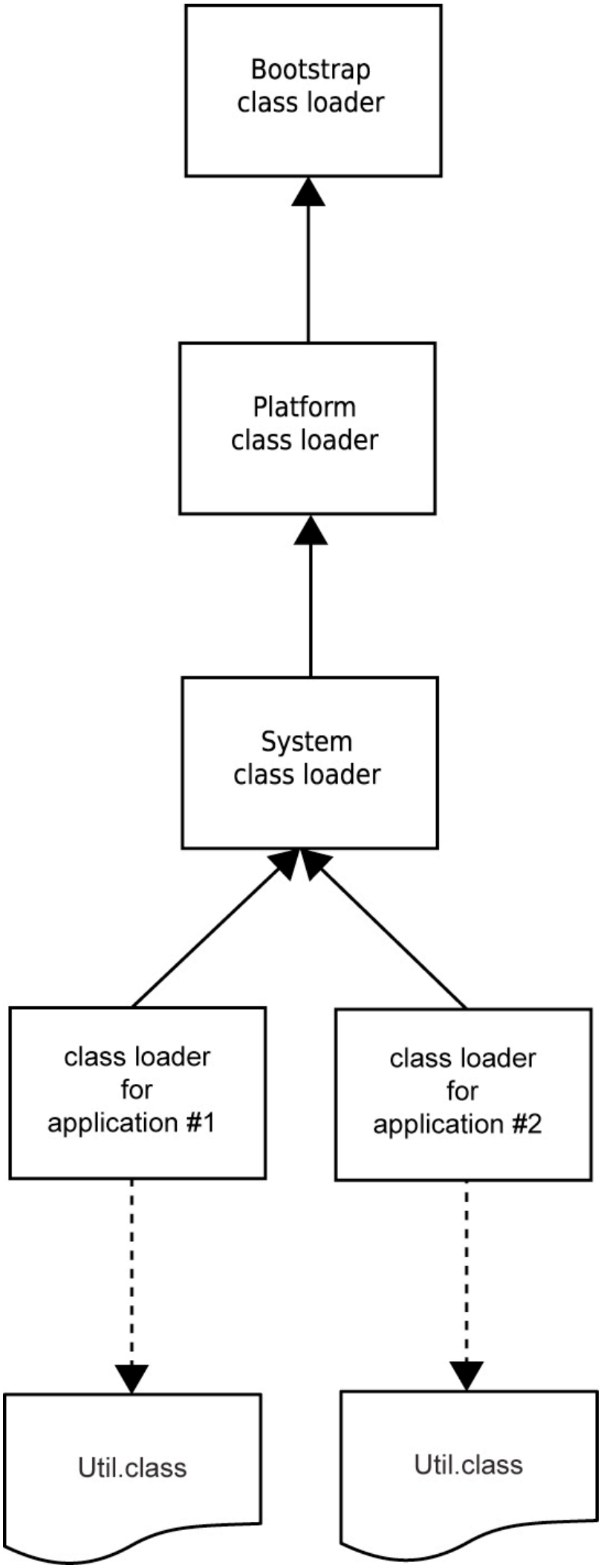


Figure 9.2: Two class loaders load different classes with the same name.

9.1.4. Writing Your Own Class Loader

You can write your own class loader for specialized purposes. That lets you carry out custom checks before you pass the bytecodes to the virtual machine. For example, your class loader may refuse to load a class that has not been marked as "paid for."

To write your own class loader, simply extend the `ClassLoader` class and override the method

```
findClass(String className)
```

The `loadClass` method of the `ClassLoader` superclass takes care of the delegation to the parent and calls `findClass` only if the class hasn't already been loaded and if the parent class loader was unable to load the class.

Your implementation of this method must do the following:

1. Load the bytecodes for the class from the local file system or some other source.
2. Call the `defineClass` method of the `ClassLoader` superclass to present the bytecodes to the virtual machine.

In the program of **Listing 9.1**, we implement a class loader that loads encrypted class files. The program asks the user for the name of the first class to load (that is, the class containing `main`) and the decryption key. It then uses a special class loader to load the specified class and calls the `main` method. The class loader decrypts the specified class and all nonsystem classes that are referenced by it. Finally, the program calls the `main` method of the loaded class.

For simplicity, we ignore the 2,000 years of progress in the field of cryptography and use the venerable Caesar cipher for encrypting the class files.



Note: David Kahn's wonderful book *The Codebreakers* (Macmillan, 1967, p. 84) refers to Suetonius as a historical source for the Caesar cipher. Caesar shifted the 24 letters of the Roman alphabet by 3 letters, which at the time baffled his adversaries.

When this chapter was first written, the U.S. government restricted the export of strong encryption methods. Therefore, I used Caesar's method for my example because it was clearly legal for export.

Our version of the Caesar cipher has as a key a number between 1 and 255. To decrypt, simply add that key to every byte and reduce modulo 256. The `Caesar.java` program of **Listing 9.2** carries out the encryption.

To not confuse the regular class loader, we use a different extension, `.caesar`, for the encrypted class files.

To decrypt, the class loader simply subtracts the key from every byte. In the companion code for this book, you will find four class files, encrypted with a key value of 3—the traditional choice. To run the encrypted program, you'll need the custom class loader defined in our `ClassLoaderDemo` program.

Try out the program by running it with a class name of `Calculator` and a decryption key of `3`. A calculator will pop up. But if you use a different key, the class loader will throw an exception.

Encrypting class files has a number of practical uses (provided, of course, that you use something stronger than the Caesar cipher). Without the decryption key, the class files are useless. They can neither be executed by a standard virtual machine nor readily disassembled.

This means that you can use a custom class loader to authenticate the user of the class or to ensure that a program has been paid for before it will be allowed to run. Of course, encryption is only one application of a custom class loader. You can use other types of class loaders to solve other problems—for example, storing class files in a database.

Listing 9.1 v2ch09/classLoader/ClassLoaderDemo.java

```
package v2ch09.classLoader;

import module java.base;

/**
 * This program demonstrates a custom class loader that decrypts class files.
 */
class ClassLoaderDemo {
    void main(String[] args) throws Exception {
        String className = IO.readln("Class name: ");
        int decryptionKey = Integer.parseInt(IO.readln("Decryption key: "));
        var loader = new CryptoClassLoader(decryptionKey);
        Class<?> c = loader.loadClass(className);
        Method m = c.getMethod("main", String[].class);
        m.invoke(null, (Object) new String[] {});
    }
}

/**
 * This class loader loads encrypted class files.
 */
class CryptoClassLoader extends ClassLoader {
    private int key;

    /**
     * Constructs a crypto class loader.
     * @param k the decryption key
     */
    public CryptoClassLoader(int k) {
        key = k;
    }

    protected Class<?> findClass(String name) throws ClassNotFoundException {
        try {
            byte[] classBytes = loadClassBytes(name);
            Class<?> cl = defineClass(name, classBytes, 0, classBytes.length);
            if (cl == null) throw new ClassNotFoundException(name);
            return cl;
        }
        catch (IOException e) {
            throw new ClassNotFoundException(name);
        }
    }

    /**
     * Loads and decrypt the class file bytes.
     * @param name the class name
     * @return an array with the class file bytes
     */
    private byte[] loadClassBytes(String name) throws IOException {
        String cname = name.replace(".", "/") + ".caesar";
        // Read encrypted class files from the same directory as this class loader
        byte[] bytes = getClass().getResourceAsStream(cname).readAllBytes();
        for (int i = 0; i < bytes.length; i++)
            bytes[i] = (byte) (bytes[i] - key);
```

```
56          return bytes;
57      }
58  }
```

Listing 9.2 v2ch09/classLoader/Caesar.java

```
1   package v2ch09.classLoader;
2
3   import module java.base;
4
5   /**
6    * Encrypts or decrypts a file using the Caesar cipher. Supply the input file, output file,
7    * and the encryption key (or its negative for decryption) on the command line.
8    */
9   class Caesar {
10      void main(String[] args) throws Exception {
11          try (var in = new FileInputStream(args[0]); var out = new FileOutputStream(args[1])) {
12              int key = Integer.parseInt(args[2]);
13              boolean done = false;
14              while (!done) {
15                  int ch = in.read();
16                  if (ch == -1)
17                      done = true;
18                  else
19                      out.write((byte) (ch + key));
20              }
21          }
22      }
23  }
```

`java.lang.Class` 1.0

- `ClassLoader getClassLoader()`
  gets the class loader that loaded this class.

`java.lang.ClassLoader` 1.0

- `ClassLoader getParent()` 1.2
  returns the parent class loader, or `null` if the parent class loader is the bootstrap class loader.
- `static ClassLoader getSystemClassLoader()` 1.2
  gets the system class loader—that is, the class loader that was used to load the first application class.
- `protected Class findClass(String name)` 1.2
  should be overridden by a class loader to find the bytecodes for a class and present them to the virtual machine by calling the `defineClass` method. In the name of the class, use `.` as package name separator, and don't use a `.class` suffix.
- `Class defineClass(String name, byte[] byteCodeData, int offset, int length)`
  adds a new class to the virtual machine whose bytecodes are provided in the given data range.

`java.net.URLClassLoader` 1.2

- `URLClassLoader(URL[] urls)`
- `URLClassLoader(URL[] urls, ClassLoader parent)`
  construct a class loader that loads classes from the given URLs. If a URL ends in a `/`, it is assumed to be a di-

rectory, otherwise it is assumed to be a JAR file.

`java.lang.Thread` 1.0

- `ClassLoader getContextClassLoader()` 1.2
  gets the class loader that the creator of this thread has designated as the most reasonable class loader to use when executing this thread.
- `void setContextClassLoader(ClassLoader loader)` 1.2
  sets a class loader for code in this thread to use for loading classes. If no context class loader is set explicitly when a thread is started, the parent thread's context class loader is used.

9.1.5. Bytecode Verification

When a class loader presents the bytecodes of a newly loaded Java platform class to the virtual machine, these bytecodes are first inspected by a *verifier*. The verifier checks that the instructions cannot perform actions that are obviously damaging. All classes except for platform classes are verified.

Here are some of the checks that the verifier carries out:

- Variables are initialized before they are used.
- Method calls match the types of object references.
- Rules for accessing private data and methods are not violated.
- Local variable accesses fall within the runtime stack.
- The runtime stack does not overflow.

If any of these checks fails, the class is considered corrupted and will not be loaded.



Note: If you are familiar with Gödel's theorem, you might wonder how the verifier can prove that a class file is free from type mismatches, uninitialized variables, and stack overflows. Gödel's theorem states that it is impossible to design algorithms that process a program and decide whether it has a particular property (such as being free from stack overflows). Is this a conflict between the public relations department at Oracle and the laws of logic? No—in fact, the verifier is *not* a decision algorithm in the sense of Gödel. If the verifier accepts a program, it is indeed safe. However, the verifier might reject virtual machine instructions even though they would actually be safe. (You might have run into this issue when you were forced to initialize a variable with a dummy value because the verifier couldn't see that it was going to be properly initialized.)

This strict verification is an important security consideration. Accidental errors, such as uninitialized variables, can easily wreak havoc if they are not caught. More importantly, in the wide open world of the Internet, you must be protected against malicious programmers who create evil effects on purpose. For example, by modifying values on the runtime stack or by writing to the private instance fields of platform objects, a program can break through the security system of a browser.

You might wonder, however, why a special verifier is needed to check all these features. After all, the compiler would never allow you to generate a class file in which an uninitialized variable is used or in which a private instance field is accessed from another class. Indeed, a class file generated by a compiler for the Java programming language always passes verification. However, the bytecode format used in the class files is well documented, and it is an easy matter for someone with experience in assembly programming and a hex editor to manually produce a class file containing valid but unsafe instructions for the Java virtual machine. The verifier is always guarding against maliciously altered class files—not just checking the class files produced by a compiler.

Here's an example of how to construct such an altered class file. We start with the program `VerifierDemo.java` of **Listing 9.3**. This is a simple program that calls a method and displays the method's result. The `fun` method itself just computes 1 + 2.

```
static int fun() {
    int m;
    int n;
    m = 1;
    n = 2;
    int r = m + n;
    return r;
}
```

As an experiment, try to compile the following modification of this program:

```
static int fun() {
    int m = 1;
    int n;
    m = 1;
    m = 2;
    int r = m + n;
    return r;
}
```

Here, `n` is not initialized, so it could have any random value. Of course, the compiler detects that problem and refuses to compile the program. To create a bad class file, we have to work a little harder. First, run the `javap` program to find out how the compiler translates the `fun` method. The command

```
javap -c v2ch09.verifier.VerifierDemo
```

shows the bytecodes in the class file in mnemonic form.

```
Method int fun()
    0 iconst_1
    1 istore_0
    2 iconst_2
    3 istore_1
    4 iload_0
    5 iload_1
    6 iadd
    7 istore_2
    8 iload_2
    9 ireturn
```

Use a hex editor to change instruction 3 from `istore_1` to `istore_0`. That is, local variable 0 (which is `m`) is initialized twice, and local variable 1 (which is `n`) is not initialized at all. We need to know the hexadecimal values for these instructions; these values are readily available from the Java Virtual Machine specification (**https://docs.oracle.com/javase/specs/jvms/se21/html/index.html**).

```
0 iconst_1 04
1 istore_0 3B
2 iconst_2 05
3 istore_1 3C
4 iload_0  1A
5 iload_1  1B
6 iadd     60
```

```
7 istore_2 3D
8 iload_2  1C
9 ireturn  AC
```

You can use any hex editor to carry out the modification. In **Figure 9.3**, you see the class file `VerifierDemo.class` loaded into the Gnome hex editor, with the bytecodes of the `fun` method highlighted.

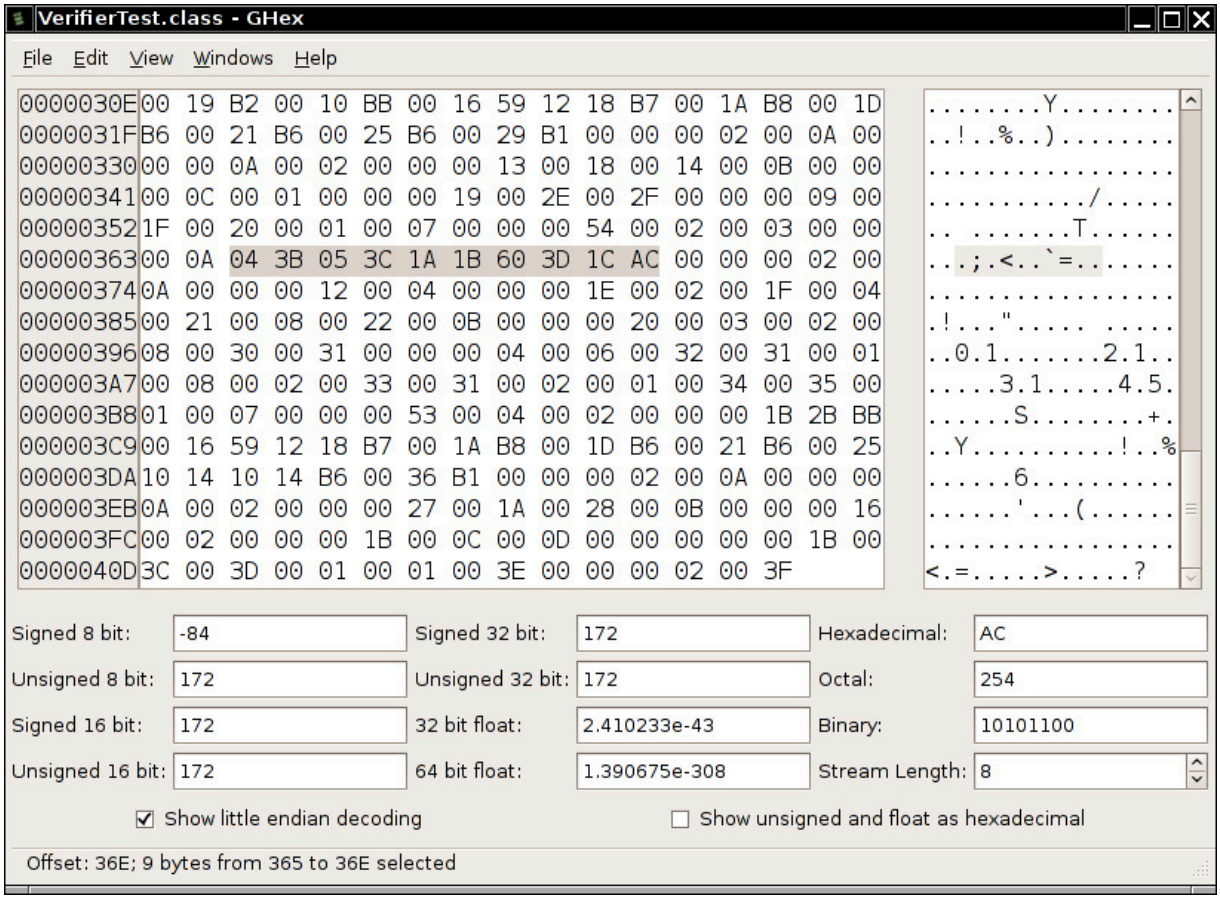


Figure 9.3: Modifying bytecodes with a hex editor

Change `3C` to `3B` and save the class file. Then try running the `VerifierDemo` program. You get an error message:

```
Exception in thread "main" java.lang.VerifyError:
    (class: VerifierDemo, method:fun signature: ()I)
    Accessing value from uninitialized register 1
```

That is good—the virtual machine detected our modification.

Now run the program with the `-noverify` (or `-Xverify:none`) option:

```
java -noverify v2ch09.verifier.VerifierDemo
```

The `fun` method returns a seemingly random value. This is actually `2` plus the value that happened to be stored in the variable `n`, which was never initialized.

Here is a typical printout:

```
1 + 2 == 15102330
```



Note: The `-noverify` and `-Xverify:none` options are deprecated and may be removed, but while they are available, they show the impact of the corruption.

Listing 9.3 v2ch09/verifier/VerifierDemo.java

```
1  package v2ch09.verifier;
2
3  /**
4   * This application demonstrates the bytecode verifier of the virtual machine. If you use a hex
5   * editor to modify the class file, then the virtual machine should detect the tampering.
6   */
7  class VerifierDemo {
8      public static void main(String[] args) {
9          System.out.println("1 + 2 == " + fun());
10     }
11
12     /**
13      * A function that computes 1 + 2.
14      * @return 3, if the code has not been corrupted
15      */
16     public static int fun() {
17         int m;
18         int n;
19         m = 1;
20         n = 2;
21         // use hex editor to change to "m = 2" in class file
22         int r = m + n;
23         return r;
24     }
25 }
```

9.2. User Authentication

The Java API provides a framework, called the Java Authentication and Authorization Service (JAAS), that provides access to platform-provided and custom authentication mechanisms. We'll discuss the JAAS framework in the following sections.

9.2.1. The JAAS Framework

As you can tell from its name, the JAAS framework has two components. The “authentication” part is concerned with ascertaining the identity of a program user. The “authorization” part is tied to the deprecated security manager, and we will not discuss it.

JAAS is a “pluggable” API that isolates Java applications from the particular technology used to implement authentication. It supports, among others, UNIX logins, Windows logins, Kerberos authentication, and certificate-based authentication.

Here is the basic outline of the login code:

```
try {
    var context = new LoginContext("Login1"); // defined in JAAS configuration file
    context.login();
    // get the authenticated Subject
    Subject subject = context.getSubject();
    . . .
    context.logout();
```

```
}
catch (LoginException e) { // thrown if login was not successful
    exception.printStackTrace();
}
```

Now the `subject` denotes the individual who has been authenticated.

The string parameter `"Login1"` in the `LoginContext` constructor refers to an entry with the same name in the JAAS configuration file. Here is a sample configuration file:

```
Login1 {
    com.sun.security.auth.module.UnixLoginModule required;
    com.whizzbang.auth.module.RetinaScanModule sufficient debug="true";
};

Login2 {
    . . .
};

other {
    . . .
};
```

The module class name and flag can be followed by any number of options that are passed on to the constructor of the module.

The special name `other` is used when the name of the login context doesn't match any of the names in the configuration.

Of course, the JDK contains no biometric login modules. The following modules are supplied in the `com.sun.security.auth.module` package:

```
UnixLoginModule
NTLoginModule
Krb5LoginModule
JndiLoginModule
KeyStoreLoginModule
```

A login policy consists of a sequence of login modules, each labeled `required`, `sufficient`, `requisite`, or `optional`. The meaning of these keywords is given by the following algorithm:

1. The modules are executed in turn, until a `sufficient` module succeeds, a `requisite` module fails, or the end of the module list is reached.
2. Authentication is successful if all `required` and `requisite` modules succeed, or if none of them were executed, if at least one `sufficient` or `optional` module succeeds.

A login authenticates a *subject*, which can have multiple *principals*. A principal describes some property of the subject, such as the username, group ID, or role. The `com.sun.security.auth.UnixPrincipal` describes the UNIX login name, and the `UnixNumericGroupPrincipal` can test for membership in a UNIX group.

The program in **Listing 9.4** displays the principals of the currently logged-in user. Run the program as

```
java -Djava.security.auth.login.config=v2ch09/auth/jaas.config v2ch09.auth.AuthDemo
```

Listing 9.5 shows the login configuration.

On Windows, change `UnixLoginModule` to `NTLoginModule` in `jaas.config`.

Listing 9.4 v2ch09/auth/AuthDemo.java

```
1  package v2ch09.auth;
2
3  import module java.base;
4
5  /**
6   * This program obtains information about a user's Unix login.
7   */
8  class AuthDemo {
9      void main() {
10         try {
11             var context = new LoginContext("Login1");
12             context.login();
13             IO.println("Authentication successful.");
14             Subject subject = context.getSubject();
15             for (Principal p : subject.getPrincipals()) {
16                 IO.println(p.getClass().getName() + ": " + p.getName());
17             }
18             context.logout();
19         }
20         catch (LoginException e) {
21             IO.println("Authentication failed.");
22             e.printStackTrace();
23         }
24     }
25 }
```

Listing 9.5 v2ch09/auth/jaas.config

```
1  Login1
2  {
3     com.sun.security.auth.module.UnixLoginModule required;
4  };
```

`javax.security.auth.login.LoginContext` 1.4

- `LoginContext(String name)`
  constructs a login context. The `name` corresponds to the login descriptor in the JAAS configuration file.
- `void login()`
  establishes a login or throws `LoginException` if the login failed. Invokes the `login` method on the modules in the JAAS configuration file.
- `void logout()`
  logs out the subject. Invokes the `logout` method on the modules in the JAAS configuration file.
- `Subject getSubject()`
  returns the authenticated subject.

`javax.security.auth.Subject` 1.4

- `Set<Principal> getPrincipals()`
  gets the principals of this subject.

java.security.Principal 1.1

- `String getName()`
  returns the identifying name of this principal.

9.2.2. JAAS Login Modules

In this section, we'll look at a JAAS example that shows you

- How to implement your own login module
- How to implement role-based authorization

Supplying your own login module is useful if you store login information in a database.

One job of the login module is to populate the principal set of the subject that is being authenticated. If a login module supports roles, it adds `Principal` objects that describe roles. The Java library does not provide a class for this purpose, so I wrote my own (see **Listing 9.6**).

The login module looks up users, passwords, and roles in a text file that contains lines like this:

```
harry|secret|admin
carl|guessme|HR
```

Of course, in a realistic login module, you would store this information in a database or directory. And you would hash the passwords—see **Section 9.3.2**.

You can find the code for the `SimpleLoginModule` in **Listing 9.7**. The `checkLogin` method checks whether the username and password match a record in the password file. If so, we add two `Principal` objects to the subject's principal set:

```
Set<Principal> principals = subject.getPrincipals();
principals.add(new UserPrincipal(username));
principals.add(new RolePrincipal(role));
```

The `UserPrincipal` class is declared in the `com.sun.security.auth` package.

The remainder of `SimpleLoginModule` is straightforward plumbing. The `initialize` method receives

- The `Subject` that is being authenticated
- A handler to retrieve login information
- A `sharedState` map that can be used for communication between login modules
- An `options` map that contains the name/value pairs that are set in the login configuration

For example, we configure our module as follows:

```
SimpleLoginModule required passwordFile="jaas/password.txt" debug=true;
```

The `initialize` method retrieves the `passwordFile` and `debug` settings from the `options` parameter.

The login module does not gather the username and password; that is the job of a separate handler. This separation allows you to use the same login module without worrying whether the login information comes from a GUI dialog box, a console prompt, or a configuration file.

The handler is specified when you construct the `LoginContext`, for example:

```
var context = new LoginContext("Login1",
    new com.sun.security.auth.callback.TextCallbackHandler());
```

The `TextCallbackHandler` class prompts for username and password from the console.

However, you usually have your own user interface for collecting the username and password. In this case, the callback mechanism is overkill, but there is no other way to transfer that information. The sample application in **Listing 9.8** uses a simple handler that merely stores and returns that information.

The handler has a single method, `handle`, that processes an array of `Callback` objects. A number of predefined classes, such as `NameCallback` and `PasswordCallback`, implement the `Callback` interface. You could also add your own class, such as `RetinaScanCallback`. The handler code is a bit unsightly because it needs to analyze the types of the callback objects:

```
public void handle(Callback[] callbacks) {
    for (Callback callback : callbacks) {
        switch (callback) {
            case NameCallback c -> c.setName(username);
            case PasswordCallback c -> c.setPassword(password);
            default -> {}
        }
    }
}
```

The `login` method of the `SimpleLoginModule` prepares an array of the callbacks that it needs for authentication:

```
var nameCallback = new NameCallback("username: ");
var passwordCallback = new PasswordCallback("password: ", false);
callbackHandler.handle(new Callback[] { nameCallback, passwordCallback });
```

Depending on the handler, the `handle` method can ask the user or retrieve the information from somewhere. When the `handle` method returns, the `nameCallback` and `passwordCallback` objects have been set. Then the `login` method retrieves the username and password and checks whether they are valid.

The program in **Listing 9.9** uses either the `TextCallbackHandler` or a custom login, with the username on the command line and a prompt for the password. When the login is successful, the principals are shown. Run the program as

```
java -Djava.security.auth.login.config=v2ch09/jaas/jaas.config v2ch09.jaas.JAASDemo
```

or

```
java -Djava.security.auth.login.config=v2ch09/jaas/jaas.config v2ch09.jaas.JAASDemo harry
```



Note: It is possible to support a more complex two-phase protocol, whereby a login is *committed* if all modules in the login configuration were successful. For more information, see the login module developer's guide at **https://docs.oracle.com/javase/8/docs/technotes/guides/security/jaas/JAASLMDevGuide.html**.

Listing 9.6 v2ch09/jaas/RolePrincipal.java

```
1  package v2ch09.jaas;
2
3  import module java.base;
4
5  /**
6   * A principal with a named role.
7   */
8  public record RolePrincipal(String name) implements Principal {
9      /**
10      * @return the role name.
11      */
12     public String getName() {
13         return name;
14     }
15 }
```

Listing 9.7 v2ch09/jaas/SimpleLoginModule.java

```
1  package v2ch09.jaas;
2
3  import module java.base;
4  import com.sun.security.auth.UserPrincipal;
5
6  /**
7   * This login module authenticates users by reading usernames, passwords, and roles from a text
8   * file.
9   */
10 public class SimpleLoginModule implements LoginModule {
11     private Subject subject;
12     private CallbackHandler callbackHandler;
13     private boolean debug;
14     private String passwordFile;
15
16     public void initialize(Subject subject, CallbackHandler callbackHandler,
17             Map<String, ?> sharedState, Map<String, ?> options) {
18         this.subject = subject;
19         this.callbackHandler = callbackHandler;
20         debug = options.get("debug").equals("true");
21         passwordFile = "" + options.get("passwordFile");
22     }
23
24     public boolean login() throws LoginException {
25         var nameCallback = new NameCallback("username: ");
26         var passwordCallback = new PasswordCallback("password: ", false);
27         try {
28             if (debug) {
29                 System.Logger logger = System.getLogger("com.horstmann.corejava");
30                 logger.log(System.Logger.Level.INFO, "Invoking handler");
31             }
32             if (callbackHandler == null) throw new LoginException("no handler");
33
34             callbackHandler.handle(new Callback[] { nameCallback, passwordCallback });
```

```
            return checkLogin(nameCallback.getName(), passwordCallback.getPassword());
        }
        catch (UnsupportedCallbackException | IOException e) {
            var e2 = new LoginException();
            e2.initCause(e);
            throw e2;
        }
    }

    /**
     * Checks whether the authentication information is valid. If it is, the subject acquires
     * principals for the user name and role.
     * @param username the user name
     * @param password a character array containing the password
     * @return true if the authentication information is valid
     */
    private boolean checkLogin(String username, char[] password) throws IOException {
        try (var in = new Scanner(Path.of(passwordFile))) {
            while (in.hasNextLine()) {
                String[] inputs = in.nextLine().split("\\|");
                if (inputs[0].equals(username)
                        && Arrays.equals(inputs[1].toCharArray(), password)) {
                    String role = inputs[2];
                    Set<Principal> principals = subject.getPrincipals();
                    principals.add(new UserPrincipal(username));
                    principals.add(new RolePrincipal(role));
                    return true;
                }
            }
            return false;
        }
    }

    public boolean logout() {
        return true;
    }

    public boolean abort() {
        return true;
    }

    public boolean commit() {
        return true;
    }
}
```

Listing 9.8 v2ch09/jaas/SimpleCallbackHandler.java

```
package v2ch09.jaas;

import module java.base;

/**
 * This simple callback handler presents the given user name and password.
 */
public class SimpleCallbackHandler implements CallbackHandler {
    private String username;
    private char[] password;

```

```
    /**
     * Constructs the callback handler.
     * @param username the user name
     * @param password a character array containing the password
     */
    public SimpleCallbackHandler(String username, char[] password) {
        this.username = username;
        this.password = password;
    }

    public void handle(Callback[] callbacks) {
        for (Callback callback : callbacks) {
            switch (callback) {
                case NameCallback c -> c.setName(username);
                case PasswordCallback c -> c.setPassword(password);
                default -> {
                }
            }
        }
    }
}
```

Listing 9.9 v2ch09/jaas/JAASDemo.java

```
package v2ch09.jaas;

import module java.base;
import module jdk.security.auth;

/**
 * This program shows how to authenticate a user via a custom login
 */
class JAASDemo {
    void main(String[] args) {
        CallbackHandler handler;
        if (args.length == 0)
            handler = new TextCallbackHandler();
        else {
            // In most apps, you have your own way of reading the username and password.
            // Here, the user provides the user name as a command line argument and the
            // password on the console.
            String username = args[0];
            Console console = System.console();
            char[] password = console.readPassword("Password: ");
            handler = new SimpleCallbackHandler(username, password);
        }

        try {
            var context = new LoginContext("Login1", handler);
            context.login();
            Subject subject = context.getSubject();
            for (Principal p : subject.getPrincipals()) {
                IO.println(p.getClass().getName() + " " + p.getName());
            }
            context.logout();
        }
        catch (LoginException e) {
            e.printStackTrace();
            Throwable cause = e.getCause();
```

```
            if (cause != null) cause.printStackTrace();
        }
    }
}
```

Listing 9.10 v2ch09/jaas/jaas.config

```
Login1
{
   jaas.SimpleLoginModule required passwordFile="jaas/password.txt" debug=true;
};
```

javax.security.auth.callback.CallbackHandler **1.4**

- `void handle(Callback[] callbacks)`
  handles the given callbacks, interacting with the user if desired, and stores the security information in the callback objects.

`javax.security.auth.callback.NameCallback` **1.4**

- `NameCallback(String prompt)`
- `NameCallback(String prompt, String defaultName)`
  construct a `NameCallback` with the given prompt and default name.
- `String getName()`
- `void setName(String name)`
  get or set the name gathered by this callback.
- `String getPrompt()`
  gets the prompt to use when querying this name.
- `String getDefaultName()`
  gets the default name to use when querying this name.

`javax.security.auth.callback.PasswordCallback` **1.4**

- `PasswordCallback(String prompt, boolean echoOn)`
  constructs a `PasswordCallback` with the given prompt and a flag indicating whether the password should be displayed as it is typed.
- `char[] getPassword()`
- `void setPassword(char[] password)`
  get or set the password gathered by this callback.
- `String getPrompt()`
  gets the prompt to use when querying this password.
- `boolean isEchoOn()`
  gets the echo flag to use when querying this password.

javax.security.auth.spi.LoginModule **1.4**

- `void initialize(Subject subject, CallbackHandler handler, Map<String,?> sharedState, Map<String,?> options)`
  initializes this `LoginModule` for authenticating the given `subject`. During login processing, uses the given handler to gather login information. Use the `sharedState` map for communicating with other login mod-

ules. The `options` map contains the name/value pairs specified in the login configuration for this module instance.
- `boolean login()`
  carries out the authentication process and populates the subject's principals. Returns `true` if the login was successful.
- `boolean commit()`
  is called after all login modules were successful, for login scenarios that require a two-phase commit. Returns `true` if the operation was successful.
- `boolean abort()`
  is called if the failure of another login module caused the login process to abort. Returns `true` if the operation was successful.
- `boolean logout()`
  logs out this subject. Returns `true` if the operation was successful.

9.3. Digital Signatures

As already mentioned, applets were what started the Java craze. In practice, people discovered that although they could write animated applets (like the famous "nervous text"), applets could not do a whole lot of useful stuff in the JDK 1.0 security model. For example, since applets under JDK 1.0 were so closely supervised, they couldn't do much good on a corporate intranet, even though relatively little risk attaches to executing an applet from your company's secure intranet. It quickly became clear to Sun that for applets to become truly useful, users need to be able to assign *different* levels of security, depending on where the applet originated. If an applet comes from a trusted supplier and has not been tampered with, the user of that applet can decide whether to give the applet more privileges.

To give more trust to an applet, we need to know two things:

- Where did the applet come from?
- Was the code corrupted in transit?

In the past 50 years, mathematicians and computer scientists have developed sophisticated algorithms for ensuring the integrity of data and for creating electronic signatures. The `java.security` package contains implementations of many of these algorithms. Fortunately, you don't need to understand the underlying mathematics to use the algorithms in the `java.security` package. In the next sections, I'll show you how message digests can detect changes in data files and how digital signatures can prove the identity of the signer.

9.3.1. Message Digests

A message digest is a digital fingerprint of a block of data. For example, the so-called SHA-1 (Secure Hash Algorithm #1) condenses any data block, no matter how long, into a sequence of 160 bits (20 bytes). As with real fingerprints, one hopes that no two different messages have the same SHA-1 fingerprint. Of course, that cannot be true—there are only 2^{160} SHA-1 fingerprints, so there must be some messages with the same fingerprint. But 2^{160} is so large that the probability of a collision is negligible. How negligible? According to James Walsh in *True Odds: How Risks Affect Your Everyday Life* (Merritt Publishing, 1996), the chance that you will die from being struck by lightning is about one in 30,000. Now, think of nine other people—for example, your nine least favorite managers or professors. The chance that you and *all of them* will die from lightning strikes is higher than that of a forged message having the same SHA-1 fingerprint as the original. (Of course, more than ten people, none of whom you are likely to know, *will* die from lightning strikes. However, we are talking about the far slimmer chance that *your particular choice* of people will be wiped out.)

A message digest has two essential properties:

- If one bit or several bits of the data are changed, the message digest also changes.

- A forger who is in possession of a given message cannot construct a fake message that has the same message digest as the original.

The second property is, again, a matter of probabilities. Consider the following message by the billionaire father:

> *"Upon my death, my property shall be divided equally among my children; however, my son George shall receive nothing."*

That message (with a final newline) has an SHA-1 fingerprint of

```
12 5F 09 03 E7 31 30 19 2E A6 E7 E4 90 43 84 B4 38 99 8F 67
```

The distrustful father has deposited the message with one attorney and the fingerprint with another. Now, suppose George bribes the lawyer holding the message. He wants to change the message so that Bill gets nothing. Of course, that changes the fingerprint to a completely different bit pattern:

```
7D F6 AB 08 EB 40 EC CD AB 74 ED E9 86 F9 ED 99 D1 45 B1 57
```

Can George find some other wording that matches the fingerprint? If he had been the proud owner of a billion computers from the time the Earth was formed, each computing a million messages a second, he would not yet have found a message he could substitute.

A number of algorithms have been designed to compute such message digests. Among them are SHA-1, the secure hash algorithm developed by the National Institute of Standards and Technology, and MD5, an algorithm invented by Ronald Rivest of MIT. Both algorithms scramble the bits of a message in ingenious ways. For details about these algorithms, see, for example, *Cryptography and Network Security, Seventh Edition*, by William Stallings (Pearson, 2017). However, subtle regularities have been discovered in both algorithms, and NIST recommends to switch to stronger alternatives. Java supports the SHA-2 and SHA-3 sets of algorithms.

The `MessageDigest` class is a *factory* for creating objects that encapsulate the fingerprinting algorithms. It has a static method, called `getInstance`, that returns an object of a class that extends the `MessageDigest` class. This means the `MessageDigest` class serves double duty:

- As a factory class
- As the superclass for all message digest algorithms

For example, here is how you obtain an object that can compute SHA fingerprints:

```
MessageDigest alg = MessageDigest.getInstance("SHA-1");
```

After you have obtained a `MessageDigest` object, feed it all the bytes in the message by repeatedly calling the `update` method. For example, the following code passes all bytes in a file to the `alg` object just created to do the fingerprinting:

```
InputStream in = . . .;
int ch;
while ((ch = in.read()) != -1)
    alg.update((byte) ch);
```

Alternatively, if you have the bytes in an array, you can update the entire array at once:

```
byte[] bytes = . . .;
alg.update(bytes);
```

When you are done, call the `digest` method. This method pads the input as required by the fingerprinting algorithm, does the computation, and returns the digest as an array of bytes.

```
byte[] hash = alg.digest();
```

The program in **Listing 9.11** computes a message digest. You can specify the file and algorithm on the command line:

```
java v2ch09.hash.Digest v2ch09/hash/input.txt SHA-1
```

If you do not supply command-line arguments, you will be prompted for the file and algorithm name.

Listing 9.11 v2ch09/hash/Digest.java

```
1  package v2ch09.hash;
2
3  import module java.base;
4
5  /**
6   * This program computes the message digest of a file.
7   */
8  class Digest {
9      /**
10      * @param args args[0] is the filename, args[1] is optionally the algorithm
11      * (SHA-1, SHA-256, or MD5)
12      */
13     void main(String[] args) throws Exception {
14         String filename = args.length >= 1 ? args[0] : IO.readln("File name: ");
15         String algname;
16         if (args.length >= 2)
17             algname = args[1];
18         else {
19             IO.println("Select one of the following algorithms: ");
20             for (Provider p : Security.getProviders())
21                 for (Provider.Service s : p.getServices())
22                     if (s.getType().equals("MessageDigest")) IO.println(s.getAlgorithm());
23             algname = IO.readln("Algorithm: ");
24         }
25         MessageDigest alg = MessageDigest.getInstance(algname);
26         byte[] input = Files.readAllBytes(Path.of(filename));
27         byte[] hash = alg.digest(input);
28         for (byte h : hash)
29             IO.print("%02X ".formatted(h & 0xFF));
30         IO.println();
31     }
32 }
```

`java.security.MessageDigest` 1.1

- `static MessageDigest getInstance(String algorithmName)`
  returns a `MessageDigest` object that implements the specified algorithm. Throws `NoSuchAlgorithmException` if the algorithm is not provided.
- `void update(byte input)`
- `void update(byte[] input)`
- `void update(byte[] input, int offset, int len)`
  update the digest, using the specified bytes.
- `byte[] digest()`
  completes the hash computation, returns the computed digest, and resets the algorithm object.
- `void reset()`
  resets the digest.

9.3.2. Password Hashing

For obvious security reasons, one should never store passwords as plain text. Instead, their hashed values should be stored. When a user logs in, the user's password entry should also be hashed. If the values match, the user is authenticated.

However, the hash algorithms that you have seen in the preceding section are not suitable for password hashing. In order to thwart brute force attacks, the hashing mechanism should use many rounds of computations. Also, to avoid precomputing of password tables, each password should be modified with a random “salt” value.

Java provides the PBKDF2 algorithm for password hashing. The algorithm has four parameters:

- A `char` array holding the password
- An array of random salt bytes
- The number of iterations
- The number of bits for the output

A salted password isn't a message, so you don't use the `MessageDigest` interface. Instead, use a `SecretKeyFactory` like this:

```
var spec = new PBEKeySpec(password, salt, ITERATIONS, 8 * HASH_BYTES);
SecretKeyFactory skf = SecretKeyFactory.getInstance("PBKDF2WithHmacSHA256");
byte[] hash = skf.generateSecret(spec).getEncoded();
```

The acronym PBE denotes “password based encryption” because the derived bytes could also be used as an encryption key. PBKDF is a “password based key derivation function”, which turns a short password into a longer encryption key. We are not interested in encryption and just store the resulting bytes. For password verification, it is also important to store the salt. Since it is good practice to increase the number of iterations over time, as processors beccome more powerful, you should store that value as well.

When storing the hash as a string, the bytes should be encoded in hex or, more efficiently, in base64:

```
Base64.Encoder encoder = Base64.getEncoder();
String hashedPassword = ITERATIONS + "|" + encoder.encodeToString(salt) + "|"
    + encoder.encodeToString(hash);
```

To verify a user-entered password, obtain the iterations and salt from the stored hash, and recompute the hash.

9.3.3. Message Signing

In **Section 9.3.1**, you saw how to compute a message digest—a fingerprint for a message. If the message is altered, the fingerprint of the altered message will not match the fingerprint of the original. If the message and its fingerprint are delivered separately, the recipient can check whether the message has been tampered with. However, if both the message and the fingerprint were intercepted, it is an easy matter to modify the message and then recompute the fingerprint. After all, the message digest algorithms are publicly known, and they don't require secret keys. In that case, the recipient of the forged message and the recomputed fingerprint would never know that the message has been altered. Digital signatures solve this problem.

To help you understand how digital signatures work, let's look at a few concepts from the field called *public key cryptography*. Public key cryptography is based on the notion of a *public* key and *private* key. The idea is that you tell everyone in the world your public key. However, only you hold the private key, and it is important that you safeguard it and don't release it to anyone else. The keys are matched by mathematical relationships, though the exact nature of these relationships is not important to us. (If you are interested, look it up in *The Handbook of Applied Cryptography* at **https://cacr.uwaterloo.ca/hac/**.)

The keys are quite long and complex. For example, here is a matching pair of public and private Digital Signature Algorithm (DSA) keys.

Public key:

```
p: fca682ce8e12caba26efccf7110e526db078b05edecbcd1eb4a208f3ae1617ae01f35b91a47e6df63413c5e12
ed0899bcd132acd50d99151bdc43ee737592e17

q: 962eddcc369cba8ebb260ee6b6a126d9346e38c5

g: 678471b27a9cf44ee91a49c5147db1a9aaf244f05a434d6486931d2d14271b9e35030b71fd73da179069b32e2
935630e1c2062354d0da20a6c416e50be794ca4

y: c0b6e67b4ac098eb1a32c5f8c4c1f0e7e6fb9d832532e27d0bdab9ca2d2a8123ce5a8018b8161a760480fadd0
40b927281ddb22cb9bc4df596d7de4d1b977d50
```

Private key:

```
p: fca682ce8e12caba26efccf7110e526db078b05edecbcd1eb4a208f3ae1617ae01f35b91a47e6df63413c5e12
ed0899bcd132acd50d99151bdc43ee737592e17

q: 962eddcc369cba8ebb260ee6b6a126d9346e38c5

g: 678471b27a9cf44ee91a49c5147db1a9aaf244f05a434d6486931d2d14271b9e35030b71fd73da179069b32e2
935630e1c2062354d0da20a6c416e50be794ca4

x: 146c09f881656cc6c51f27ea6c3a91b85ed1d70a
```

It is believed to be practically impossible to compute one key from the other. That is, even though everyone knows your public key, they can't, in your lifetime, compute your private key, no matter how many computing resources they have available.

It may be difficult to believe that the private key can't be computed from the public key—but nobody has found an algorithm to do this for the encryption algorithms in common use today. If the keys are sufficiently long, brute force—simply trying all possible keys—would require more computers than can be built from all the atoms in the solar system, crunching away for thousands of years. Of course it is possible that someone could come up with algorithms for computing keys that are much more clever than brute force. For example, the RSA algorithm (the encryption algo-

rithm invented by Rivest, Shamir, and Adleman) depends on the difficulty of factoring large numbers. For the last 20 years, many of the best mathematicians have tried to come up with good factoring algorithms, but so far with no success. For that reason, most cryptographers believe that keys with a "modulus" of 2,000 bits or more are currently completely safe from any attack. DSA is believed to be similarly secure.

Figure 9.4 illustrates how the process works in practice.

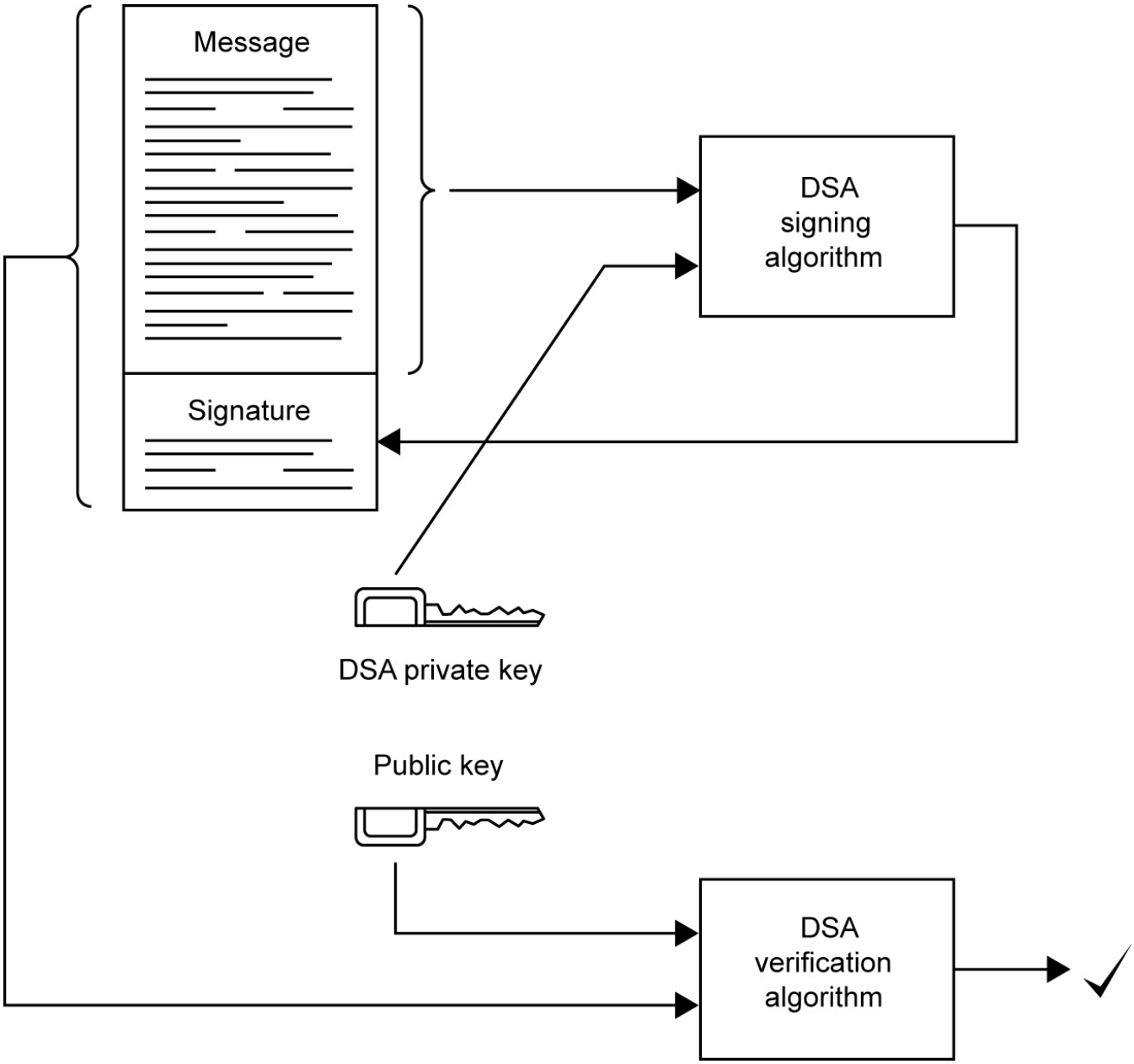


Figure 9.4: Public key signature exchange with DSA

Suppose Alice wants to send Bob a message, and Bob wants to know this message came from Alice and not an impostor. Alice writes the message and *signs* the message digest with her private key. Bob gets a copy of her public key. Bob then applies the public key to *verify* the signature. If the verification passes, Bob can be assured of two facts:

- The original message has not been altered.
- The message was signed by Alice, the holder of the private key that matches the public key that Bob used for verification.

You can see why the security of private keys is so important. If someone steals Alice's private key, or if a government can require her to turn it over, then she is in trouble. The thief or a government agent can now impersonate her by sending messages, such as money transfer instructions, that others will believe came from Alice.

9.3.4. Verifying a Signature

The JDK comes with the `keytool` program, which is a command-line tool to generate and manage a set of certificates. We expect that ultimately the functionality of this tool will be embedded in other, more user-friendly programs. But right now, we'll use `keytool` to show how Alice can sign a document and send it to Bob, and how Bob can verify that the document really was signed by Alice and not an impostor.

The `keytool` program manages *keystores*—databases of certificates and private/public key pairs. Each entry in the keystore has an *alias*. Here is how Alice creates a keystore, `alice.jks`, and generates a key pair with alias `alice`:

```
keytool -genkeypair -keyalg DSA -keystore alice.jks -alias alice
```

When creating or opening a keystore, you are prompted for a keystore password. For this example, just use `secret`. If you were to use the `keytool`-generated keystore for any serious purpose, you would need to choose a good password and safeguard this file.

When generating a key, you are prompted for the following information:

```
Enter keystore password:  secret
Reenter new password:  secret
What is your first and last name?
  [Unknown]:  Alice Lee
What is the name of your organizational unit?
  [Unknown]:  Engineering
What is the name of your organization?
  [Unknown]:  ACME Software
What is the name of your City or Locality?
  [Unknown]:  San Francisco
What is the name of your State or Province?
  [Unknown]:  CA
What is the two-letter country code for this unit?
  [Unknown]:  US
Is <CN=Alice Lee, OU=Engineering, O=ACME Software, L=San Francisco, ST=CA, C=US> correct?
  [no]:  yes
```

The `keytool` uses names in the X.500 format, whose components are Common Name (CN), Organizational Unit (OU), Organization (O), Location (L), State (ST), and Country (C), to identify key owners and certificate issuers.

Suppose Alice wants to give her public key to Bob. She needs to export a certificate file:

```
keytool -exportcert -keystore alice.jks -alias alice -file alice.cer
```

Now Alice can send the certificate to Bob. When Bob receives the certificate, he can print it:

```
keytool -printcert -file alice.cer
```

The printout looks like this:

```
Owner: CN=Alice Lee, OU=Engineering, O=ACME Software, L=San Francisco, ST=CA, C=US
Issuer: CN=Alice Lee, OU=Engineering, O=ACME Software, L=San Francisco, ST=CA, C=US
Serial number: b2b46ccd99479da2
Valid from: Tue Jan 16 08:11:22 CET 2024 until: Mon Apr 15 09:11:22 CEST 2024
```

```
Certificate fingerprints:
  SHA1: 41:38:E6:F2:9D:FA:90:FE:CB:36:85:F1:E6:39:62:A1:5A:DD:16:BC
  SHA256: 14:CC:AD:BC:83:D5:0C:F9:98:24:52:29:B3:AF:B2:B9:05:A7:C0:61:15:63:EA:FB:13:4F:73:37:D2:FD:8D:FB
Signature algorithm name: SHA256withDSA
Subject Public Key Algorithm: 2048-bit DSA key
Version: 3
```

If Bob wants to check that he got the right certificate, he can call Alice and verify the certificate fingerprint over the phone.



Note: Some certificate issuers publish certificate fingerprints on their web sites. For example, to check the DigiCert certificate in the keystore `var jdk` /lib/security/cacerts

directory, use the `-list` option:

```
keytool -list -v -keystore $JAVA_HOME/lib/security/cacerts
```

The password for this keystore is `changeit`. One of the certificates in this keystore is

```
Owner: CN=DigiCert Assured ID Root G3, OU=www.digicert.com, O=DigiCert Inc, C=US
Issuer: CN=DigiCert Assured ID Root G3, OU=www.digicert.com, O=DigiCert Inc, C=US
Serial number: ba15afa1ddfa0b54944afcd24a06cec
Valid from: Thu Aug 01 14:00:00 CEST 2013 until: Fri Jan 15 13:00:00 CET 2038
Certificate fingerprints:
         SHA1: F5:17:A2:4F:9A:48:C6:C9:F8:A2:00:26:9F:DC:0F:48:2C:AB:30:89
         SHA256: 7E:37:CB:8B:4C:47:09:0C:AB:36:55:1B:A6:F4:5D:B8:40:68:0F:BA:
                 16:6A:95:2D:B1:00:71:7F:43:05:3F:C2
```

You can check that your certificate is valid by visiting the web site **https://www.digicert.com/kb/digicert-root-certificates.htm**.

Once Bob trusts the certificate, he can import it into his keystore.

```
keytool -importcert -keystore bob.jks -alias alice -file alice.cer
```



Caution: Never import into a keystore a certificate that you don't fully trust. Once a certificate is added to the keystore, any program that uses the keystore assumes that the certificate can be used to verify signatures.

Now Alice can start sending signed documents to Bob. The `jarsigner` tool signs and verifies JAR files. Alice simply adds the document to be signed into a JAR file.

```
jar cvf document.jar document.txt
```

She then uses the `jarsigner` tool to add the signature to the file. She needs to specify the keystore, the JAR file, and the alias of the key to use.

```
jarsigner -keystore alice.jks document.jar alice
```

When Bob receives the file, he uses the `-verify` option of the `jarsigner` program.

```
jarsigner -verify -keystore bob.jks document.jar
```

Bob does not need to specify the key alias. The `jarsigner` program finds the X.500 name of the key owner in the digital signature and looks for a matching certificate in the keystore.

If the JAR file is not corrupted and the signature matches, the `jarsigner` program prints

```
jar verified.
```

followed by a number of warnings that you can ignore for this simple experiment. Otherwise, the program displays an error message.



The KeyStore Explorer program (**https://keystore-explorer.org/**) provides a graphical user interface for working with keystores.

9.3.5. The Authentication Problem

Suppose you get a message from your friend Alice, signed with her private key, using the method I just showed you. You might already have her public key, or you can easily get it by asking her for a copy or by getting it from her web page. Then, you can verify that the message was in fact authored by Alice and has not been tampered with. Now, suppose you get a message from a stranger who claims to represent a famous software company, urging you to run a program attached to the message. The stranger even sends you a copy of his public key so you can verify that he authored the message. You check that the signature is valid. This proves that the message was signed with the matching private key and has not been corrupted.

Be careful: *You still have no idea who wrote the message.* Anyone can generate a pair of public and private keys, sign the message with the private key, and send the signed message and the public key to you. The problem of determining the identity of the sender is called the *authentication problem.*

The usual way to solve the authentication problem is simple. Suppose the stranger and you have a common acquaintance you both trust. Suppose the stranger meets your acquaintance in person and hands over a USB stick with the public key. Your acquaintance later meets you, assures you that he met the stranger and that the stranger indeed works for the famous software company, and then gives you the USB stick (see **Figure 9.5**). That way, your acquaintance vouches for the authenticity of the stranger.

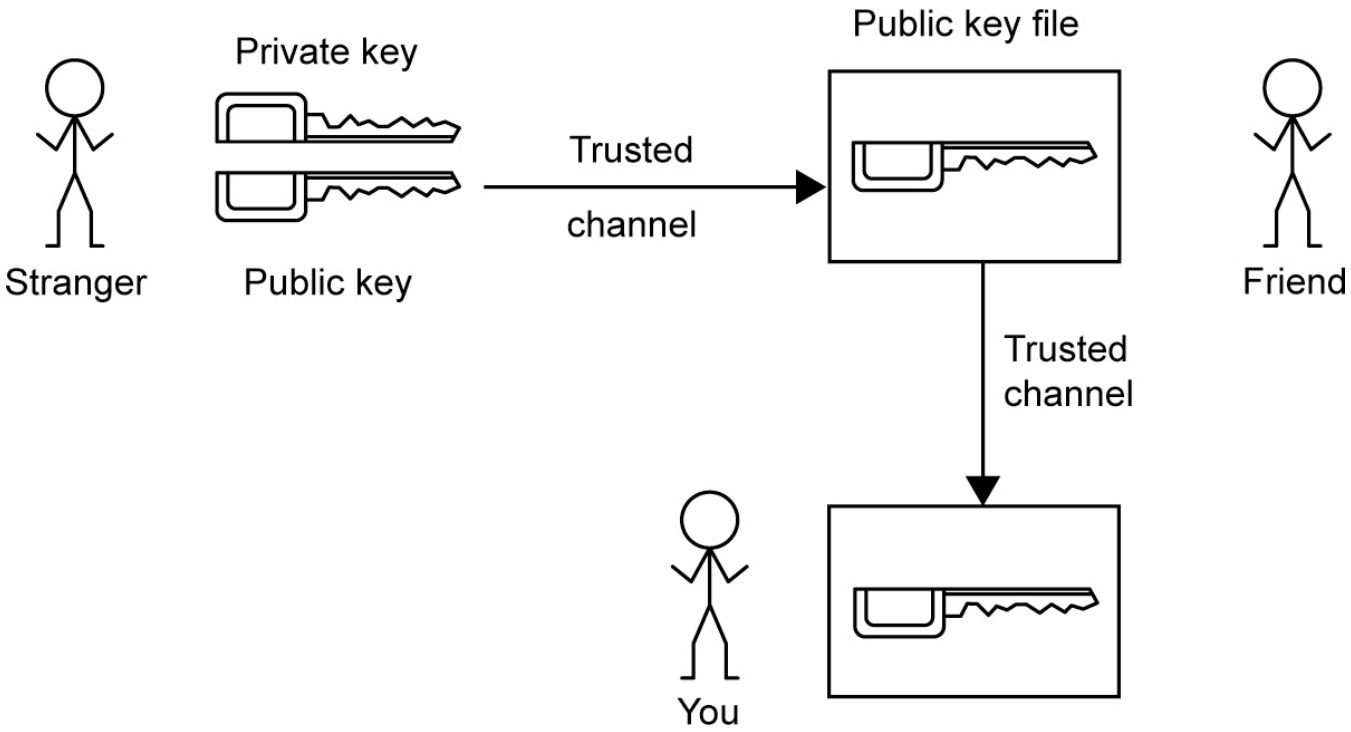


Figure 9.5: Authentication through a trusted intermediary

In fact, your acquaintance does not actually need to meet you. Instead, he can use his private key to sign the stranger's public key file (see **Figure 9.6**).

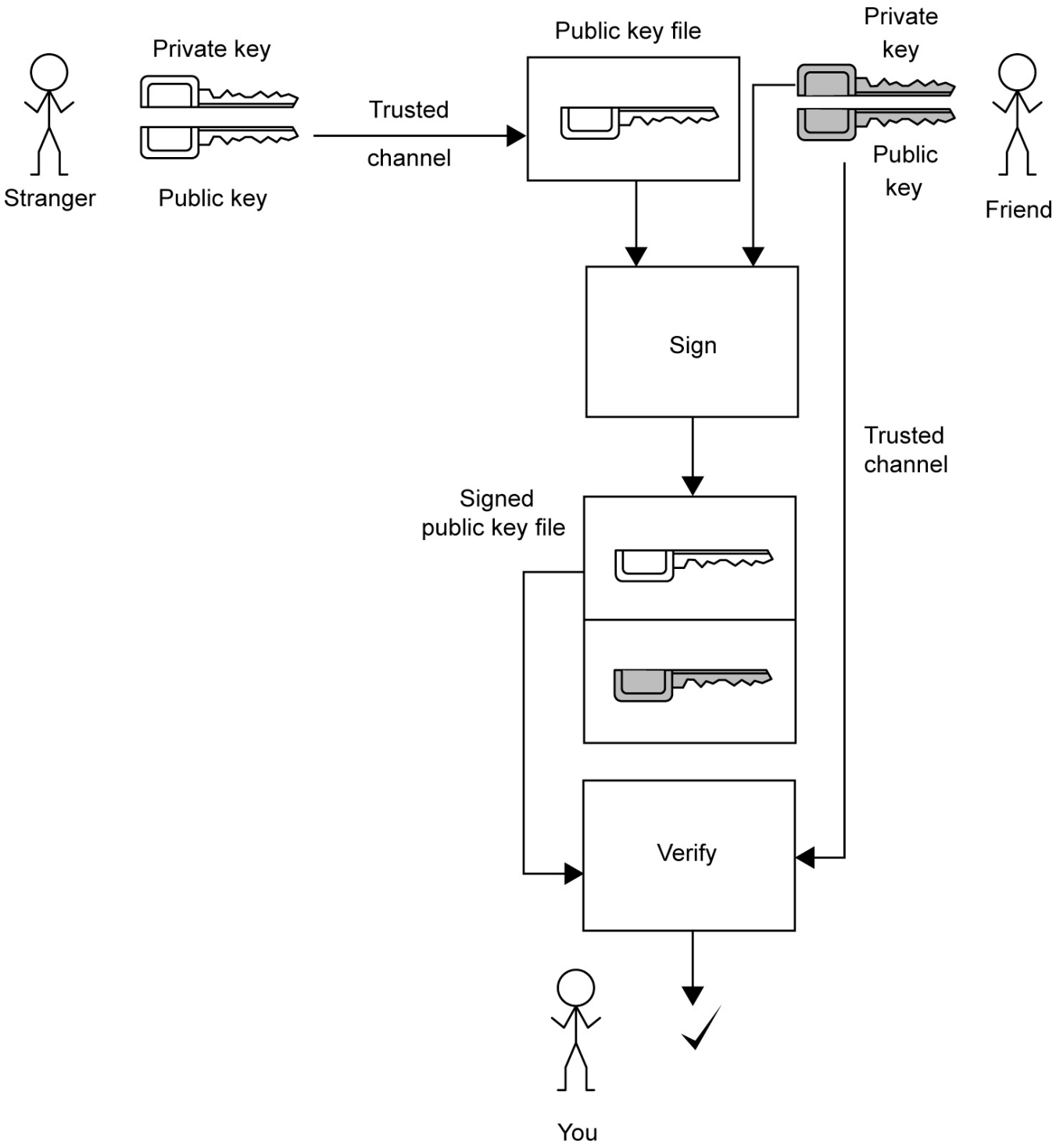


Figure 9.6: Authentication through a trusted intermediary's signature

When you get the public key file, you verify the signature of your friend, and because you trust him, you are confident that he did check the stranger's credentials before applying his signature.

However, you might not have a common acquaintance. Some trust models assume that there is always a "chain of trust"—a chain of mutual acquaintances—so that you trust every member of that chain. In practice, of course, that isn't always true. You might trust your friend, Alice, and you know that Alice trusts Bob, but you don't know Bob and aren't sure that you trust him. Other trust models assume that there is a benevolent big brother—a company in which we all trust. Companies with confidence-inspiring names such as DigiCert, GlobalSign, and Entrust provide verification services.

You will often encounter digital signatures signed by one or more entities who will vouch for the authenticity, and you will need to evaluate to what degree you trust the authenticators. You might place a great deal of trust in a particular certificate authority, perhaps because you saw their logo on many web pages or because you heard that they require multiple people with black attaché cases to come together into a secure chamber whenever new master keys are to be minted.

However, you should have realistic expectations about what is actually being authenticated. You can get a “class 1” ID simply by filling out a web form and paying a small fee. The key is mailed to the e-mail address included in the certificate. Thus, you can be reasonably assured that the e-mail address is genuine, but the requestor could have filled in *any* name and organization. There are more stringent classes of IDs. For example, with a “class 3” ID, the certificate authority will require an individual requestor to appear before a notary public, and it will check the financial rating of a corporate requestor. Other authenticators will have different procedures. Thus, when you receive an authenticated message, it is important that you understand what, in fact, is being authenticated.

9.3.6. Certificate Signing

In **Section 9.3.4** you saw how Alice used a certificate to distribute a public key to Bob. However, Bob needed to ensure that the certificate was valid by verifying the fingerprint with Alice.

Suppose Alice wants to send her colleague Cindy a signed message, but Cindy doesn’t want to bother with verifying lots of signature fingerprints. Now suppose there is an entity that Cindy trusts to verify signatures. In this example, Cindy trusts the Information Resources Department at ACME Software.

That department operates a *certificate authority* (CA). Everyone at ACME has the CA’s public key in their keystore, installed by a system administrator who carefully checked the key fingerprint. The CA signs the keys of ACME employees. When they install each other’s keys, the keystore will trust them implicitly because they are signed by a trusted key.

Here is how you can simulate this process. Create a keystore `acmesoft.jks`. Generate a key pair and export the public key:

```
keytool -genkeypair -keystore acmesoft.jks -keyalg DSA -alias acmeroot
keytool -exportcert -keystore acmesoft.jks -alias acmeroot -file acmeroot.cer
```

The public key is exported into a “self-signed” certificate. Then, add it to every employee’s keystore:

```
keytool -importcert -keystore cindy.jks -alias acmeroot -file acmeroot.cer
```

For Alice to send messages to Cindy and to everyone else at ACME Software, she needs to bring a signing request for her certificate to the Information Resources Department, and have her key signed. Here are the steps:

```
keytool -keystore alice.jks -alias alice -certreq -file alice.req
keytool -storepass secret -keystore acmesoft.jks -gencert -alias acmeroot \
    -infile alice.req -outfile alice_signed_by_acmeroot.cer
```

Alice gives the file `alice_signedby_acmeroot.cer` to Cindy and to anyone else in ACME Software. Alternatively, ACME Software can simply store the file in a company directory. Remember, this file contains Alice’s public key and an assertion by ACME Software that this key really belongs to Alice.

Now Cindy imports the signed certificate into her keystore:

```
keytool -importcert -keystore cindy.jks -alias alice -file alice_signedby_acmeroot.cer
```

The keystore verifies that the key was signed by a trusted root key that is already present in the keystore. Cindy is *not* asked to verify the certificate fingerprint.

Once Cindy has added the root certificate and the certificates of the people who regularly send her documents, she does not have to worry about the certicicates again until they expire.

9.3.7. Code Signing

A common use of authentication technology is signing executable programs. If you download a program, you are naturally concerned about the damage it can do. For example, the program could have been infected by a virus. If you know where the code comes from *and* that it has not been tampered with since it left its origin, your comfort level will be a lot higher than without this knowledge.

In this section, I'll show you how to sign JAR files, and how you can configure Java to verify the signature. This capability was designed for applets and Java Web Start applications. These are no longer commonly used technologies, but you may still need to support them in legacy products.

When Java was first released, applets ran in the "sandbox," with limited permissions, as soon as they were loaded. If users wanted to use applets that can access the local file system, make network connections, and so on, they had to explicitly agree. To ensure that the applet code was not tampered with in transit, it had to be digitally signed.

Here is a specific example. Suppose that while surfing the Internet, you encounter a web site that offers to run an applet from an unfamiliar vendor, provided you grant it the permission to do so (see **Figure 9.7**). Such a program is signed with a *software developer* certificate issued by a certificate authority that the Java runtime trusts. The pop-up dialog box identifies the software developer and the certificate issuer. Now you need to decide whether to authorize the program.

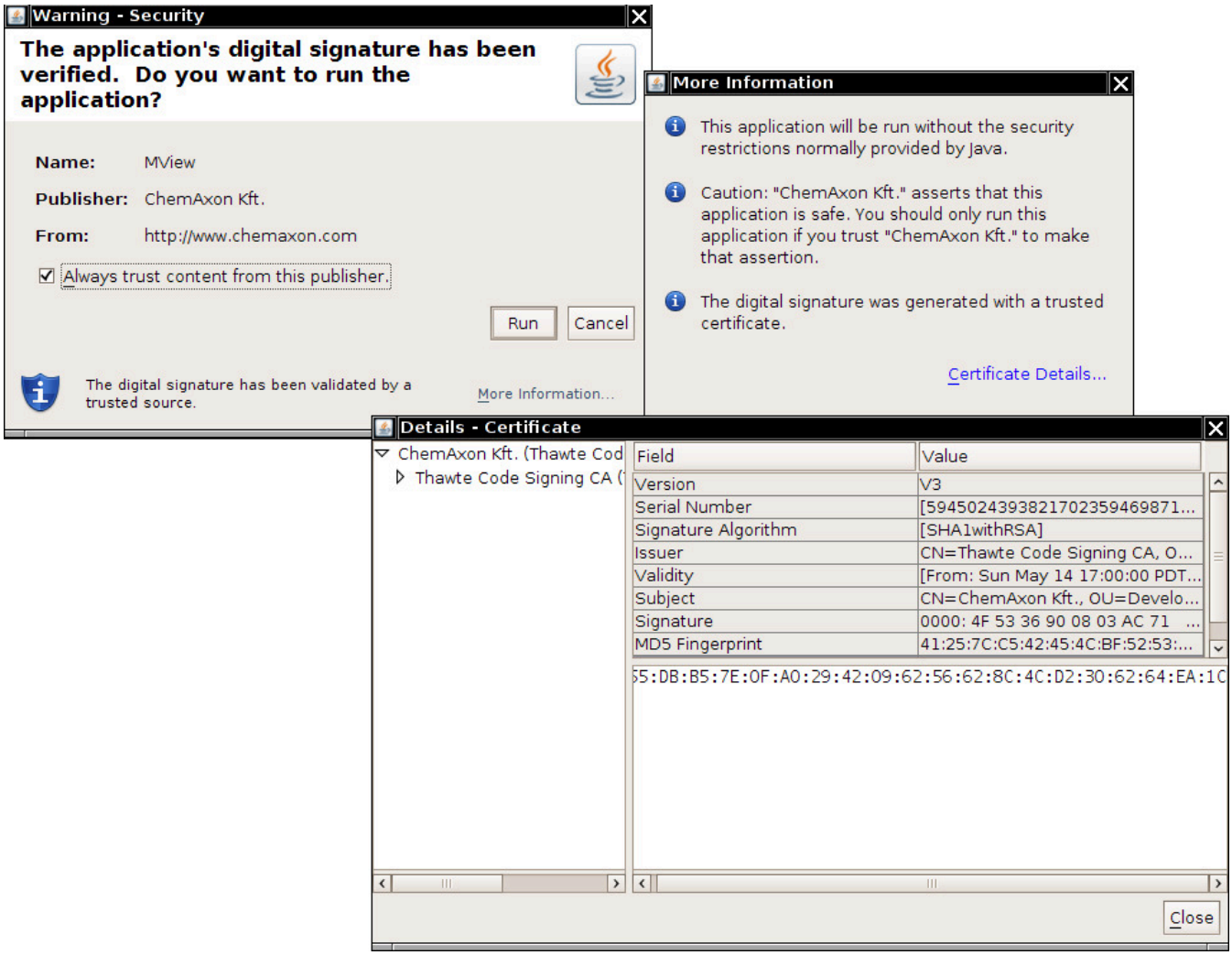


Figure 9.7: Launching a signed applet

What facts do you have at your disposal that might influence your decision? Here is what you know:

- Thawte sold a certificate to the software developer.
- The program really was signed with that certificate, and it hasn't been modified in transit.
- That certificate really was signed by Thawte—it was verified by the public key in the local `cacerts` file.

Of course, none of this tells you whether the code is safe to run. Can you trust a vendor if all you know is the vendor's name and the fact that Thawte sold them a software developer certificate? This approach never made much sense.

For intranet deployment, certificates are more plausible. Administrators can install policy files and certificates on local machines so that no user interaction is required for launching trusted code. However, with the deprecation of the security manager, this approach is no longer viable.

9.4. Encryption

So far, we have discussed one important cryptographic technique implemented in the Java security API—namely, authentication through digital signatures. A second important aspect of security is *encryption*. Even when authenticated, the information itself is plainly visible. The digital signature merely verifies that the information has not been changed. In contrast, when information is encrypted, it is not visible. It can only be decrypted with a matching key.

Authentication is sufficient for code signing—there is no need to hide the code. However, encryption is necessary when applications transfer confidential information, such as credit card numbers and other personal data.

In the past, patents and export controls prevented many companies from offering strong encryption. Fortunately, export controls are now much less stringent, and the patents for important algorithms have expired. Nowadays, Java provides excellent encryption support as a part of the standard library.

9.4.1. Symmetric Ciphers

The Java cryptographic extensions contain a class `Cipher` that is the superclass of all encryption algorithms. To get a cipher object, call the `getInstance` method:

```
Cipher cipher = Cipher.getInstance(algorithmName);
```

or

```
Cipher cipher = Cipher.getInstance(algorithmName, providerName);
```

The JDK comes with ciphers by the provider named `"SunJCE"`. It is the default provider used if you don't specify another provider name. You might want another provider if you need specialized algorithms that Oracle does not support.

The algorithm name is a string such as `"AES"` or `"DES/CBC/PKCS5Padding"`.

The Data Encryption Standard (DES) is a venerable block cipher with a key length of 56 bits. Nowadays, the DES algorithm is considered obsolete because it can be cracked with brute force. A far better alternative is its successor, the Advanced Encryption Standard (AES). See **https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197.pdf** for a detailed description of the AES algorithm. I use AES in this example.

Once you have a cipher object, initialize it by setting the mode and the key:

```
int mode = . . .;
Key key = . . .;
cipher.init(mode, key);
```

The mode is one of

```
Cipher.ENCRYPT_MODE
Cipher.DECRYPT_MODE
Cipher.WRAP_MODE
Cipher.UNWRAP_MODE
```

The wrap and unwrap modes encrypt one key with another—see the next section for an example.

Now you can repeatedly call the `update` method to encrypt blocks of data:

```
int blockSize = cipher.getBlockSize();
var inBytes = new byte[blockSize];
. . . // read inBytes
int outputSize= cipher.getOutputSize(blockSize);
var outBytes = new byte[outputSize];
int outLength = cipher.update(inBytes, 0, outputSize, outBytes);
. . . // write outBytes
```

When you are done, you must call the `doFinal` method once. If a final block of input data is available (with fewer than `blockSize` bytes), call

```
outBytes = cipher.doFinal(inBytes, 0, inLength);
```

If all input data have been encrypted, instead call

```
outBytes = cipher.doFinal();
```

The call to `doFinal` is necessary to carry out *padding* of the final block. Consider the DES cipher. It has a block size of eight bytes. Suppose the last block of the input data has fewer than eight bytes. Of course, we can fill the remaining bytes with `0`, to obtain one final block of eight bytes, and encrypt it. But when the blocks are decrypted, the result will have several trailing `0` bytes appended to it, and therefore will be slightly different from the original input file. To avoid this problem, we need a *padding scheme*. A commonly used padding scheme is the one described in the Public Key Cryptography Standard (PKCS) #5 by RSA Security, Inc. (**https://datatracker.ietf.org/doc/html/rfc2898**).

In this scheme, the last block is not padded with a pad value of zero, but with a pad value that equals the number of pad bytes. In other words, if `L` is the last (incomplete) block, it is padded as follows:

```
L 01                              if length(L) = 7
L 02 02                           if length(L) = 6
L 03 03 03                        if length(L) = 5
. . .
L 07 07 07 07 07 07 07            if length(L) = 1
```

Finally, if the length of the input is actually divisible by 8, then one block

```
08 08 08 08 08 08 08 08
```

is appended to the input and encrypted. After decryption, the very last byte of the plaintext is a count of the padding characters to discard.

9.4.2. Key Generation

To encrypt, you need to generate a key. Each cipher has a different format for keys, and you need to make sure that the key generation is random. Follow these steps:

1. Get a `KeyGenerator` for your algorithm.
2. Initialize the generator with a source for randomness. If the block length of the cipher is variable, also specify the desired block length.
3. Call the `generateKey` method.

For example, here is how you generate an AES key:

```
KeyGenerator keygen = KeyGenerator.getInstance("AES");
var random = new SecureRandom(); // see below
keygen.init(random);
Key key = keygen.generateKey();
```

Alternatively, you can produce a key from a fixed set of raw data (perhaps derived from a password or the timing of keystrokes). Construct a `SecretKeySpec` (which implements the `SecretKey` interface) like this:

```
byte[] keyData = . . .; // 16 bytes for AES
var key = new SecretKeySpec(keyData, "AES");
```

When generating keys, make sure you use *truly random* numbers. For example, the regular random number generator in the `Random` class, seeded by the current date and time, is not random enough. Suppose the computer clock is accurate to 1/10 of a second. Then there are at most 864,000 seeds per day. If an attacker knows the day a key was issued (which can often be deduced from a message date or certificate expiration date), it is an easy matter to generate all possible seeds for that day.

The `SecureRandom` class generates random numbers that are far more secure than those produced by the `Random` class. You still need to provide a seed to start the number sequence at a random spot. The best method for doing this is to obtain random input from a hardware device such as a white-noise generator. Another reasonable source for random input is to ask the user to type away aimlessly on the keyboard, with each keystroke contributing only one or two bits to the random seed. Once you gather such random bits in an array of bytes, pass it to the `setSeed` method:

```
var random = new SecureRandom();
var b = new byte[20];
// fill with truly random bits
random.setSeed(b);
```

If you don't seed the random number generator, it will compute its own 20-byte seed by launching threads, putting them to sleep, and measuring the exact time when they are awakened.



Note: This algorithm is *not* known to be safe. In the past, algorithms that relied on the timing of some components of the computer, such as hard disk access time, were shown not to be completely random.

The sample program at the end of this section puts the AES cipher to work (see **Listing 9.12**). The `crypt` utility method in **Listing 9.13** will be reused in other examples. To use the program, you first need to generate a secret key. Run

```
java v2ch09.aes.AESDemo -genkey secret.key
```

The secret key is saved in the file `secret.key`.

Now you can encrypt with the command

```
java v2ch09.aes.AESDemo -encrypt plaintextFile encryptedFile secret.key
```

Decrypt with the command

```
java v2ch09.aes.AESDemo -decrypt encryptedFile decryptedFile secret.key
```

The program is straightforward. The `-genkey` option produces a new secret key and serializes it in the given file. The `-encrypt` and `-decrypt` options both call into the same `crypt` method that calls the `update` and `doFinal` methods of the cipher. Note how the `update` method is called so long as the input blocks have the full length, and the `doFinal` method is either called with a partial input block (which is then padded) or with no additional data (to generate one pad block).

Listing 9.12 v2ch09/aes/AESDemo.java

```
1  package v2ch09.aes;
2
3  import module java.base;
4
5  /**
6   * This program tests the AES cipher. It can generate a secret key,
7   * or encrypt or decrypt a file.
8   */
9  class AESDemo {
10     void main(String[] args) throws Exception {
11         if (args[0].equals("-genkey")) {
12             KeyGenerator keygen = KeyGenerator.getInstance("AES");
13             var random = new SecureRandom();
14             keygen.init(random);
15             SecretKey key = keygen.generateKey();
16             try (var out = new ObjectOutputStream(new FileOutputStream(args[1]))) {
17                 out.writeObject(key);
18             }
19         }
20         else {
21             int mode;
22             if (args[0].equals("-encrypt"))
23                 mode = Cipher.ENCRYPT_MODE;
24             else
25                 mode = Cipher.DECRYPT_MODE;
26
27             try (var keyIn = new ObjectInputStream(new FileInputStream(args[3]));
28                     var in = new FileInputStream(args[1]);
29                     var out = new FileOutputStream(args[2])) {
30                 var key = (Key) keyIn.readObject();
31                 Cipher cipher = Cipher.getInstance("AES");
32                 cipher.init(mode, key);
33                 Util.crypt(in, out, cipher);
34             }
35         }
36     }
37 }
```

Listing 9.13 v2ch09/aes/Util.java

```
package v2ch09.aes;

import module java.base;

public class Util {
    /**
     * Uses a cipher to transform the bytes in an input stream and sends the transformed bytes
     * to an output stream.
     * @param in the input stream
     * @param out the output stream
     * @param cipher the cipher that transforms the bytes
     */
    public static void crypt(InputStream in, OutputStream out, Cipher cipher)
            throws IOException, GeneralSecurityException {
        int blockSize = cipher.getBlockSize();
        int outputSize = cipher.getOutputSize(blockSize);
        var inBytes = new byte[blockSize];
        var outBytes = new byte[outputSize];

        int inLength = 0;
        boolean done = false;
        while (!done) {
            inLength = in.read(inBytes);
            if (inLength == blockSize) {
                int outLength = cipher.update(inBytes, 0, blockSize, outBytes);
                out.write(outBytes, 0, outLength);
            }
            else
                done = true;
        }
        if (inLength > 0)
            outBytes = cipher.doFinal(inBytes, 0, inLength);
        else
            outBytes = cipher.doFinal();
        out.write(outBytes);
    }
}
```

`javax.crypto.Cipher` **1.4**

- `static Cipher getInstance(String algorithmName)`
- `static Cipher getInstance(String algorithmName, String providerName)`
  return a `Cipher` object that implements the specified algorithm. Throw a `NoSuchAlgorithmException` if the algorithm is not provided.
- `int getBlockSize()`
  returns the size (in bytes) of a cipher block, or `0` if the cipher is not a block cipher.
- `int getOutputSize(int inputLength)`
  returns the size of an output buffer that is needed if the next input has the given number of bytes. This method takes into account any buffered bytes in the cipher object.
- `void init(int mode, Key key)`
  initializes the cipher algorithm object. The mode is one of `ENCRYPT_MODE`, `DECRYPT_MODE`, `WRAP_MODE`, or `UNWRAP_MODE`.

- `byte[] update(byte[] in)`
- `byte[] update(byte[] in, int offset, int length)`
- `int update(byte[] in, int offset, int length, byte[] out)`
  transform one block of input data. The first two methods return the output. The third method returns the number of bytes placed into `out`.
- `byte[] doFinal()`
- `byte[] doFinal(byte[] in)`
- `byte[] doFinal(byte[] in, int offset, int length)`
- `int doFinal(byte[] in, int offset, int length, byte[] out)`
  transform the last block of input data and flush the buffer of this algorithm object. The first three methods return the output. The fourth method returns the number of bytes placed into `out`.

`javax.crypto.KeyGenerator` 1.4

- `static KeyGenerator getInstance(String algorithmName)`
  returns a `KeyGenerator` object that implements the specified algorithm. Throws a `NoSuchAlgorithmException` if the algorithm is not provided.
- `void init(SecureRandom random)`
- `void init(int keySize, SecureRandom random)`
  initialize the key generator.
- `SecretKey generateKey()`
  generates a new key.

`javax.crypto.spec.SecretKeySpec` 1.4

- `SecretKeySpec(byte[] key, String algorithmName)`
  constructs a key specification.

9.4.3. Cipher Streams

The JCE library provides a convenient set of stream classes that automatically encrypt or decrypt stream data. For example, here is how you can encrypt data to a file:

```
Cipher cipher = . . .;
cipher.init(Cipher.ENCRYPT_MODE, key);
var out = new CipherOutputStream(new FileOutputStream(outputFileName), cipher);
var bytes = new byte[BLOCKSIZE];
int inLength = getData(bytes); // get data from data source
while (inLength != -1) {
     out.write(bytes, 0, inLength);
     inLength = getData(bytes); // get more data from data source
}
out.flush();
```

Similarly, you can use a `CipherInputStream` to read and decrypt data from a file:

```
Cipher cipher = . . .;
cipher.init(Cipher.DECRYPT_MODE, key);
var in = new CipherInputStream(new FileInputStream(inputFileName), cipher);
var bytes = new byte[BLOCKSIZE];
int inLength = in.read(bytes);
while (inLength != -1) {
```

```
    putData(bytes, inLength); // put data to destination
    inLength = in.read(bytes);
}
```

The cipher stream classes transparently handle the calls to `update` and `doFinal`, which is clearly a convenience.

`javax.crypto.CipherInputStream` 1.4

- `CipherInputStream(InputStream in, Cipher cipher)`
  constructs an input stream that reads data from `in` and decrypts or encrypts them by using the given cipher.
- `int read()`
- `int read(byte[] b, int off, int len)`
  read data from the input stream, which is automatically decrypted or encrypted.

`javax.crypto.CipherOutputStream` 1.4

- `CipherOutputStream(OutputStream out, Cipher cipher)`
  constructs an output stream that writes data to `out` and encrypts or decrypts them using the given cipher.
- `void write(int ch)`
- `void write(byte[] b, int off, int len)`
  write data to the output stream, which is automatically encrypted or decrypted.
- `void flush()`
  flushes the cipher buffer and carries out padding if necessary.

9.4.4. Public Key Ciphers

The AES cipher that you have seen in the preceding section is a *symmetric* cipher. The same key is used for both encryption and decryption. The Achilles heel of symmetric ciphers is key distribution. If Alice sends Bob an encrypted message, Bob needs the same key that Alice used. If Alice changes the key, she needs to send Bob both the message and, through a secure channel, the new key. But perhaps she has no secure channel to Bob—which is why she encrypts her messages to him in the first place.

Public key cryptography solves that problem. In a public key cipher, Bob has a key pair consisting of a public key and a matching private key. Bob can publish the public key anywhere, but he must closely guard the private key. Alice simply uses the public key to encrypt her messages to Bob.

Actually, it's not quite that simple. All known public key algorithms are *much* slower than symmetric key algorithms such as DES or AES. It would not be practical to use a public key algorithm to encrypt large amounts of information. However, that problem can easily be overcome by combining a public key cipher with a fast symmetric cipher, like this:

1. Alice generates a random symmetric encryption key. She uses it to encrypt her plaintext.
2. Alice encrypts the symmetric key with Bob's public key.
3. Alice sends Bob both the encrypted symmetric key and the encrypted plaintext.
4. Bob uses his private key to decrypt the symmetric key.
5. Bob uses the decrypted symmetric key to decrypt the message.

Nobody but Bob can decrypt the symmetric key because only Bob has the private key for decryption. Thus, the expensive public key encryption is only applied to a small amount of key data.

The most commonly used public key algorithm is the RSA algorithm invented by Rivest, Shamir, and Adleman. Until October 2000, the algorithm was protected by a patent assigned to RSA Security, Inc. Licenses were not cheap—typi-

cally a 3% royalty, with a minimum payment of $50,000 per year. Now the algorithm is in the public domain.

To use the RSA algorithm, you need a public/private key pair. Use a `KeyPairGenerator` like this:

```
KeyPairGenerator pairgen = KeyPairGenerator.getInstance("RSA");
var random = new SecureRandom();
pairgen.initialize(KEYSIZE, random);
KeyPair keyPair = pairgen.generateKeyPair();
Key publicKey = keyPair.getPublic();
Key privateKey = keyPair.getPrivate();
```

The program in **Listing 9.14** has three options. The `-genkey` option produces a key pair. The `-encrypt` option generates an AES key and *wraps* it with the public key.

```
Key key = . . .; // an AES key
Key publicKey = . . .; // a public RSA key
Cipher cipher = Cipher.getInstance("RSA");
cipher.init(Cipher.WRAP_MODE, publicKey);
byte[] wrappedKey = cipher.wrap(key);
```

It then produces a file that contains

- The length of the wrapped key
- The wrapped key bytes
- The plaintext encrypted with the AES key

The `-decrypt` option decrypts such a file. To try the program, first generate the RSA keys:

```
java v2ch09.rsa.RSADemo -genkey public.key private.key
```

Then encrypt a file:

```
java v2ch09.rsa.RSADemo -encrypt plaintextFile encryptedFile public.key
```

Finally, decrypt it and verify that the decrypted file matches the plaintext:

```
java v2ch09.rsa.RSADemo -decrypt encryptedFile decryptedFile private.key
```

Listing 9.14 v2ch09/rsa/RSADemo.java

```
1  package v2ch09.rsa;
2
3  import module java.base;
4
5  import v2ch09.aes.Util;
6
7  /**
8   * This program tests the RSA cipher. It can generate public and private key pairs,
9   * or encrypt or decrypt a file.
10  */
11 class RSADemo {
```

```
    final int KEYSIZE = 512;

    void main(String[] args) throws Exception {
        if (args[0].equals("-genkey")) {
            KeyPairGenerator pairgen = KeyPairGenerator.getInstance("RSA");
            var random = new SecureRandom();
            pairgen.initialize(KEYSIZE, random);
            KeyPair keyPair = pairgen.generateKeyPair();
            try (var out = new ObjectOutputStream(new FileOutputStream(args[1]))) {
                out.writeObject(keyPair.getPublic());
            }
            try (var out = new ObjectOutputStream(new FileOutputStream(args[2]))) {
                out.writeObject(keyPair.getPrivate());
            }
        }
        else if (args[0].equals("-encrypt")) {
            KeyGenerator keygen = KeyGenerator.getInstance("AES");
            var random = new SecureRandom();
            keygen.init(random);
            SecretKey key = keygen.generateKey();

            // wrap with RSA public key
            try (var keyIn = new ObjectInputStream(new FileInputStream(args[3]));
                    var out = new DataOutputStream(new FileOutputStream(args[2]));
                    var in = new FileInputStream(args[1])) {
                var publicKey = (Key) keyIn.readObject();
                Cipher cipher = Cipher.getInstance("RSA");
                cipher.init(Cipher.WRAP_MODE, publicKey);
                byte[] wrappedKey = cipher.wrap(key);
                out.writeInt(wrappedKey.length);
                out.write(wrappedKey);

                cipher = Cipher.getInstance("AES");
                cipher.init(Cipher.ENCRYPT_MODE, key);
                Util.crypt(in, out, cipher);
            }
        }
        else {
            try (var in = new DataInputStream(new FileInputStream(args[1]));
                    var keyIn = new ObjectInputStream(new FileInputStream(args[3]));
                    var out = new FileOutputStream(args[2])) {
                int length = in.readInt();
                var wrappedKey = new byte[length];
                in.read(wrappedKey, 0, length);

                // unwrap with RSA private key
                var privateKey = (Key) keyIn.readObject();

                Cipher cipher = Cipher.getInstance("RSA");
                cipher.init(Cipher.UNWRAP_MODE, privateKey);
                Key key = cipher.unwrap(wrappedKey, "AES", Cipher.SECRET_KEY);

                cipher = Cipher.getInstance("AES");
                cipher.init(Cipher.DECRYPT_MODE, key);

                Util.crypt(in, out, cipher);
            }
        }
    }
}
```

In this chapter, you had a glimpse at class loaders, and you saw the services for authentication and encryption that the Java library provides. Along the way, you perhaps shed a tear that the classic Java security architecture, based on a security manager and code signing, is now a distant memory.

In the next chapter, we will delve into advanced Swing and graphics programming.

Previous chapter
< 8. Compiling and Scripting

Next chapter
10. Graphical User Interface Programming >